

PINNs for Learning High-Frequency Elastic Waves in Complex Layered Media

Abdul Hannan Mustajab

Kiel University

Frank Wuttke

Kiel University

Ayra Riaz Khan

Aligarh Muslim University

Zarghaam Haider Rizvi

`zarghaam.rizvi@geoanalysis-engineering.de`

GeoAnalysis Engineering

Article

Keywords: pinn, piratenet, elastic wave, elastodynamics

Posted Date: November 27th, 2025

DOI: <https://doi.org/10.21203/rs.3.rs-8006794/v1>

License:   This work is licensed under a Creative Commons Attribution 4.0 International License.

[Read Full License](#)

Additional Declarations: No competing interests reported.

PINNs for Learning High-Frequency Elastic Waves in Complex Layered Media

Abdul Hannan Mustajab¹, Frank Wuttke¹, Ayra Riaz Khan²,
Zarghaam Haider Rizvi^{3*}

¹*Geomechanics and Geotechnics, Kiel University, Kiel, Germany.

²Department of Computer Engineering, Aligarh Muslim University,
Aligarh, India.

³Numerical Modelling and Physics Infused Intelligence, GeoAnalysis
Engineering, Kiel, Germany.

*Corresponding author(s). E-mail(s):

zarghaam.rizvi@geoanalysis-engineering.de;

Contributing authors: abdul-hannan.mustajab@ifg.uni-kiel.de;
frank.wuttke@ifg.uni-kiel.de; khanayrariaz@gmail.com;

Abstract

Accurately modelling high-frequency elastic waves in heterogeneous media is central to problems spanning geotechnical engineering, seismic hazard assessment, subsurface energy exploration, and planetary interior studies. Yet traditional numerical solvers are computationally intensive and often struggle with small-scale heterogeneity. Here, we show that physics-informed neural networks (PINNs) can efficiently learn the two-dimensional elastic wave equation without labelled data, using only physical residuals as constraints. We benchmark three architectures, a standard PINN, a Fourier-feature PINN, and the PirateNet, across homogeneous and multi-layered velocity models. Only PirateNet reproduces complex reflections, transmissions, and mode conversions in six-layered media, achieving stable high-frequency predictions where conventional PINNs fail. Applied examples include subsurface imaging for geothermal and nuclear-energy systems and elastic-wave analysis of planetary crusts. These results highlight the capacity of adaptive PINNs to integrate physical laws and neural representation, offering a unified framework for next-generation simulation and inversion across the Earth and planetary sciences.

Keywords: pinn, piratenet, elastic wave, elastodynamics

1 Introduction

Elastic wave equations describe the propagation of mechanical disturbances in elastic media. They play a vital role in numerous applications, including seismic exploration for hydrocarbon and mineral resources, subsurface geological characterization, and non-destructive testing in engineering contexts. Traditional methods such as finite-difference (FD), finite-element (FE), and spectral-element (SE) methods have been widely used to solve elastic wave equations [1–3]. However, these approaches often encounter challenges like mesh generation, potential numerical dispersion, and truncation errors arising from approximate stencils.

One of the most popular and flexible methods for embedding physical principles in machine learning is through the formulation of tailored loss functions. Physics-Informed Neural Networks (PINNs) (Raissi et al.) have garnered significant interest as a method for solving partial differential equations (PDEs), primarily due to their capability of directly embedding governing physical laws into the training process via specialized loss functions. Unlike traditional numerical methods, PINNs offer a mesh-free computational framework and require minimal labelled data, effectively circumventing the need for extensive data collection. Due to their inherent flexibility and straightforward implementation, PINNs have been successfully applied to both forward and inverse PDE problems, seamlessly integrating noisy experimental measurements alongside physical constraints [4, 5]. In this framework, the PDEs and their corresponding boundary or initial conditions act as constraints during model training, thus substantially reducing data dependency, a common bottleneck when modelling complex real-world phenomena. Additionally, PINNs utilize automatic differentiation for derivative computations, significantly mitigating truncation errors associated with classical numerical differentiation techniques.

Once trained, a PINN efficiently predicts solutions at any arbitrary point within the spatio-temporal domain, eliminating the need for iterative forward simulations. This capability is particularly beneficial in computationally intensive tasks such as full waveform inversion (FWI) or imaging. PINNs are poised to substantially improve computational performance and solution accuracy by embedding fundamental wave propagation principles into the neural network architecture. Our current methodology provides a robust foundation for further development in critical applications, including full waveform inversion for non-destructive evaluation and testing.

PINNs have been successfully applied in the diverse scientific and engineering fields. In geosciences, Sarma et al. introduced Interface PINNs (I-PINNs), which are designed to model interface problems characterized by sharp discontinuities. They proposed the use of separate neural networks for distinct subdomains on either side of the interface, differing exclusively by their activation functions while maintaining identical network parameters. Building upon the I-PINNS framework, Roy et al. introduced Adaptive Interface-PINNS (AdaI-PINNS), enhancing the modelling of interface problems exhibiting discontinuous coefficients and interfacial jumps. AdaI-PINNs integrate trainable slopes into their activation functions, enabling a fully adaptive, automated

framework without the necessity of predefined activations. [Moseley et al.](#) demonstrated that PINNS can solve acoustic wave equations efficiently, and later introduced a domain-decomposition method (FBPINNs) ([Moseley et al.](#)) for larger-scale problems. [Yu et al.](#) introduced Gradient enhanced PINNs (gPINNs), which were later used by [Xie et al.](#) to solve the acoustic wave equation. [Rasht-Behesht et al.](#) incorporated snapshot data for initial conditions, showing how data can be used to enhance PINN-based solutions in early time steps for the acoustic wave equation. [Mustajab et al.](#) tackled the one-dimensional (1D) acoustic wave problem, highlighting how transfer learning could mitigate the common spectral bias issue. [Alkhadhr and Almekkawy](#) investigated the effect of different boundary constraints on PINN training. [Herrmann et al.](#) proposed a hybrid approach for full waveform inversion, employing a neural network ansatz to learn material fields. Although these studies confirm PINNs can handle acoustic wave equations effectively, even under complex scenarios, applications to the elastic wave equation remain relatively limited.

A few notable exceptions include the work of [Rao et al.](#), who used PINNs to solve the elastic wave equation for homogeneous parameter distributions, and of [Liang et al.](#), who devised a frequency-domain PINN approach to address spectral bias in elastodynamics. Additionally, [Staub](#) proposed an advanced PINN architecture, embedding encoder-decoder structures and wavelet/plane wave feature layers to model wavefields with heterogeneous parameters accurately. These works collectively demonstrate the promise of PINNs for elastic wave equations, yet they also highlight the difficulties of capturing high-frequency components particularly relevant when wave speeds vary substantially or when complex boundary conditions induce large gradients. [Mu et al.](#) proposed SeparationPINN, which is a PINN framework that uses Helmholtz decomposition to derive separate wavefield terms. This provides a mesh-free numerical solution that effectively captures wavefield features at multiple scales. Numerical tests demonstrate that the proposed PINN-based separation method can accurately separate P- and S-waves in both homogeneous and heterogeneous media.

To address the spectral bias issue, [Tancik et al.](#) introduced Fourier feature mappings, which embed input coordinates into a higher-dimensional space, enabling networks to capture high-frequency components more effectively. However, training deeper networks for more intricate wave phenomena remains challenging due to issues such as vanishing or exploding gradients. Residual connections, or ResNets, have shown promise for stabilizing deep network training. A recent modification of ResNets, known as PirateNets, was introduced by [Wang et al.](#). PirateNets employ adaptive skip connections and residual learning, particularly suitable for scenarios requiring deeper architectures or more complex representations.

In this paper, we systematically assess the performance of PirateNets for solving the elastic wave equation in the time domain, with a particular focus on layered media characterized by discontinuous Lamé parameters and varying wave speeds. We compare the results against two other PINN-based methods: a fully connected neural network (FCNN) approach and a Fourier-feature PINN. Importantly, our approach remains mesh-free and data-free: we do not rely on labeled data, and we enforce the

PDE constraints through automatic differentiation. We demonstrate this across three test cases of increasing complexity: (i) a homogeneous medium, (ii) a single-layered medium, and (iii) a multi-layered medium with complex material variability.

The remainder of this paper is organized as follows. [Section 2](#) reviews the fundamental theory of elastic waves. [Section 3](#) describes our $(2 + 1)$ D PINN approach, including the design of the neural network inputs/outputs and the loss functions that enforce both the PDE and boundary/initial conditions. We then discuss various network architectures in [Section 3.3](#), where we introduce the vanilla-PINN, the Fourier-feature PINN, and the PirateNet architecture, along with the notion of hard enforcement of initial conditions. [Section 4](#) presents numerical examples for homogeneous, single-layered, and multi-layered media, comparing performance across the three neural approaches. Finally, [Section 5](#) concludes the paper and offers directions for future research.

2 Theory

2.1 Elastic Wave Propagation

In this work, we focus on solving the two-dimensional (2D) elastic wave equation for an isotropic medium using Physics-Informed Neural Networks (PINNs). In isotropic materials, the mechanical properties (mass density ρ and Lamé parameters λ and μ) are the same in all directions. A homogeneous medium further implies that these properties remain uniform throughout the domain. We solve the equation under the plane strain condition.

For an isotropic, 2D domain, the elastic wave equation can be written as (Meng and Fu)

$$\rho \frac{\partial^2 \mathbf{u}}{\partial t^2} = \nabla \cdot \boldsymbol{\sigma} + \mathbf{f}, \quad (1)$$

where ρ is the mass density, $\mathbf{u} = [u_x, u_y]^T$ is the displacement vector, $\boldsymbol{\sigma}$ is the stress tensor, $\mathbf{f} = [f_x, f_y]^T$ represents any external force per unit volume, and t is time. In this paper, the excitation is introduced through initial conditions, so we set $\mathbf{f} = \mathbf{0}$. Since we introduce the wavefield through given initial conditions in this work, the external force terms f_x and f_y are set to zero. Consequently, any source-like behaviour in the model is enforced through the initial displacement and/or velocity fields rather than through an explicit forcing term.

Under the assumption of linear elasticity for an isotropic and homogeneous medium, Hooke's law relates the stress tensor $\boldsymbol{\sigma}$ to the strain tensor $\boldsymbol{\epsilon}$:

$$\boldsymbol{\sigma} = \lambda \text{Tr}(\boldsymbol{\epsilon}) \mathbf{I} + 2\mu \boldsymbol{\epsilon}, \quad (2)$$

where λ and μ are the Lamé parameters, $\text{Tr}(\boldsymbol{\epsilon}) = \epsilon_{xx} + \epsilon_{yy}$ is the trace of the strain tensor (i.e., the volumetric strain), and $\mathbf{I}$ is the identity matrix. The strain tensor $\boldsymbol{\epsilon}$ is related to the displacement gradients by

$$\boldsymbol{\epsilon} = \begin{bmatrix} \epsilon_{xx} & \epsilon_{xy} \\ \epsilon_{xy} & \epsilon_{yy} \end{bmatrix} = \frac{1}{2} \begin{bmatrix} \frac{\partial u_x}{\partial x} & \frac{\partial u_x}{\partial y} + \frac{\partial u_y}{\partial x} \\ \frac{\partial u_x}{\partial y} + \frac{\partial u_y}{\partial x} & \frac{\partial u_y}{\partial y} \end{bmatrix}. \quad (3)$$

In this study, all physical parameters and variables, including the Lamé parameters λ , μ , and the density ρ , are expressed in non-dimensional form. The governing equations have been non-dimensionalized using characteristic scales of length, time, and displacement to simplify the numerical treatment and improve generality of the results.

We use absorbing boundary conditions (ABCs) to handle reflections at the boundaries of the computational domain. Unlike other numerical methods that require the explicit implementation of ABC operators, PINNs do not need any additional management, such as adding a separate partial differential equation (PDE) loss term near the boundaries (Rasht-Behesht et al., Sethi et al.).

P-wave and S-wave velocities

The compressional and shear wave speeds are respectively given by ([Aki and Richards](#)):

$$V_p = \sqrt{\frac{\lambda + 2\mu}{\rho}} \quad (4)$$

$$V_s = \sqrt{\frac{\mu}{\rho}} \quad (5)$$

3 Methods

This section provides details on the methodologies used to solve the 2D elastic wave equation using PINNs. We begin with an overview of PINNs, then outline the mathematical formulation and explain how the governing partial differential equation is embedded into the loss function. Next, we describe the standard neural-network architecture and training strategies. Finally, we introduce two key architectural enhancements, i.e., Fourier feature embeddings and PirateNets[20], to mitigate spectral bias and training challenges in deeper networks, thereby ensuring stable training.

3.1 PINN Formulation For Elastodynamic Wave

We consider two-dimensional, plane-strain, linear isotropic elastodynamics on a space-time domain $\Omega \times [0, T]$ with displacement $\mathbf{u}(t, \mathbf{x}) = [u_x, u_y]^\top$ and material parameters (ρ, λ, μ) . The governing equations (derived in Eq. (1)) are

$$\rho \partial_{tt} \mathbf{u} = \nabla \cdot \boldsymbol{\sigma} + \mathbf{f}, \quad \boldsymbol{\sigma} = \lambda \text{Tr}(\boldsymbol{\varepsilon}) \mathbf{I} + 2\mu \boldsymbol{\varepsilon}, \quad \boldsymbol{\varepsilon} = \frac{1}{2}(\nabla \mathbf{u} + (\nabla \mathbf{u})^\top), \quad (6)$$

where $\text{tr}(\cdot)$ denotes the trace and the divergence acts row-wise on $\boldsymbol{\sigma}$. In all experiments we excite the medium through initial conditions and therefore set $\mathbf{f} \equiv \mathbf{0}$ in the PINN formulation.

We approximate the vector field with a neural network $\hat{\mathbf{u}}(t, \mathbf{x}; \theta)$, compute all required derivatives by automatic differentiation, and minimize a physics loss given by the strong-form residual evaluated at space-time collocation points $\{(t_i, \mathbf{x}_i)\}_{i=1}^{N_{\text{PDE}}}$:

$$L_{\text{PDE}}(\theta) = \frac{1}{N_{\text{PDE}}} \sum_{i=1}^{N_{\text{PDE}}} \|\mathbf{R}(t_i, \mathbf{x}_i; \theta)\|_2^2. \quad (7)$$

$$\text{where } \mathbf{R}(t, \mathbf{x}; \theta) = \rho \partial_{tt} \hat{\mathbf{u}}(t, \mathbf{x}; \theta) - \nabla \cdot \boldsymbol{\sigma}(\hat{\mathbf{u}}(t, \mathbf{x}; \theta)),$$

$$\boldsymbol{\sigma}(\hat{\mathbf{u}}) = \lambda \text{Tr}(\hat{\boldsymbol{\varepsilon}}) \mathbf{I} + 2\mu \hat{\boldsymbol{\varepsilon}},$$

$$\hat{\boldsymbol{\varepsilon}} = \frac{1}{2}(\nabla \hat{\mathbf{u}} + (\nabla \hat{\mathbf{u}})^\top).$$

The above corresponds to the strong-form PINN residual used throughout the paper. In Fig. 1, the whole training pipeline is described. The network outputs are multiplied by an ansatz to enforce initial conditions, and the strong form residual is the only loss term we optimize.

Following the work of Staub, we use the following initial conditions:

$$\mathbf{u}(0, \mathbf{x}) = g(\mathbf{x}), \quad \partial_t \mathbf{u}(0, \mathbf{x}) = 0, \quad (8)$$

where g is built from a Gaussian f and its derivative:

$$f(\mathbf{x}) = e^{\left(-\frac{|\mathbf{x} - \mathbf{x}_0|^2}{2\sigma_g^2}\right)}, \quad \frac{df(\mathbf{x})}{d\mathbf{x}} = -\frac{\mathbf{x} f(\mathbf{x})}{\sigma_g^2}, \quad (9)$$

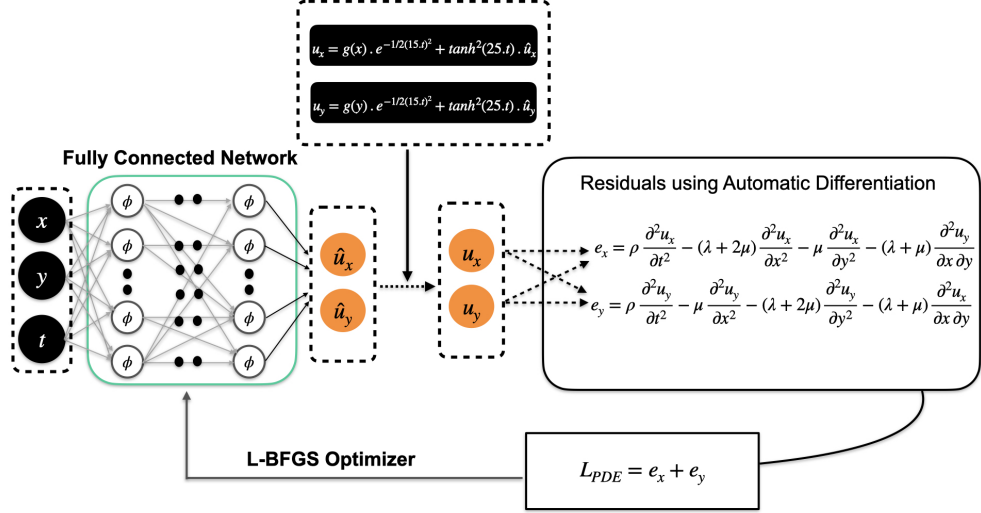


Fig. 1: Spatiotemporal inputs (x, y, t) are passed through a neural backbone (green box), which can be chosen as a multilayer perceptron (MLP), Fourier-feature network, or a PirateNet. The network outputs $\hat{u}_x$ and $\hat{u}_y$ are multiplied by an ansatz to exactly satisfy the prescribed initial conditions. The resulting fields are used to compute the elastodynamic residuals e_x and e_y via automatic differentiation. Training minimizes the mean-squared error (MSE) of the residual loss $L_{PDE} = e_x + e_y$ optimized using the L-BFGS optimizer.

and

$$g(\mathbf{x}) = \frac{\frac{df(\mathbf{x})}{d\mathbf{x}}}{\left\| \frac{df(\mathbf{x})}{d\mathbf{x}} \right\|_{\max}}. \quad (10)$$

In all experiments, we set $\sigma_g = 0.1$ and $\mathbf{x}_0 = \mathbf{0}$ (Gaussian centered in the domain). The used initial conditions can be visualized in Fig. 3.

The final expression, $\mathbf{R}(t, \mathbf{x}; \theta)$, is the strong-form residual of the governing PDE. A key aspect of our methodology is that the training loss consists only of this physics-based residual. We can adopt this simplified loss function because we handle the initial conditions directly, rather than adding them as penalty terms. For enforcing the initial conditions, the neural network's architecture is modified to automatically satisfy them. This technique is detailed in Section 3.2.

3.2 Hard Imposition of Initial Conditions

In standard PINNs, initial conditions are enforced weakly via penalty terms, which do not guarantee exact satisfaction. Following [8, 17, 22], we enforce the initial conditions strictly by embedding them in the solution ansatz. Rather than learning the solution directly, we use the ansatz

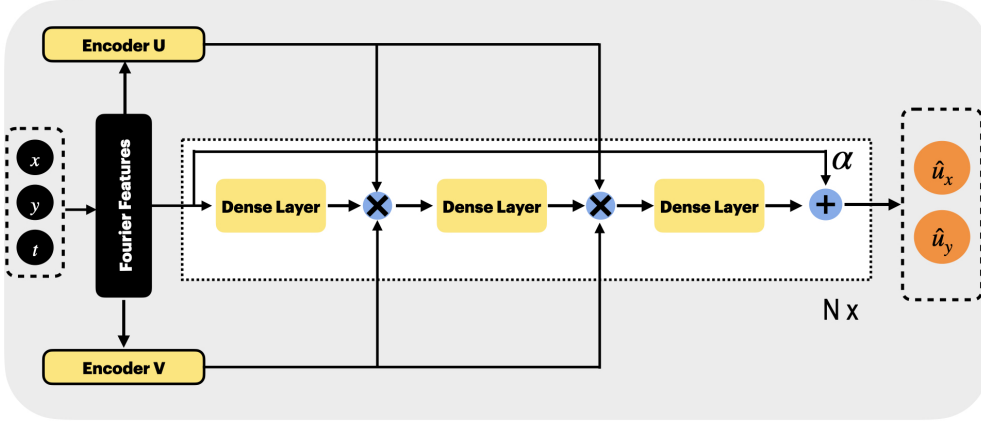


Fig. 2: Spatiotemporal inputs (t, x, y) are mapped into a higher-dimensional space using random Fourier features enhanced by auxiliary encoders $\mathbf{U}$ and $\mathbf{V}$. The features are processed through N adaptive residual blocks, each comprising three dense layers and gated skip connections that integrate shallow representations. A trainable parameter α , initialized at zero, ensures that the blocks act as identity mappings at the start of training, allowing the network to be interpreted as a linear combination of coordinate embeddings. As training proceeds, α progressively activates, enabling the model to deepen adaptively and refine predictions of the displacement fields $\hat{u}_x$ and $\hat{u}_y$.

$$\hat{u}(\mathbf{x}, t; \theta) = g(\mathbf{x}) \cdot e^{-1/2(15 \cdot t)^2} + \tanh^2(25 \cdot t) \cdot NN(\mathbf{x}, t; \theta) \quad (11)$$

to define the approximate solution in the PINN optimization problem. It is straightforward to verify that this ansatz automatically satisfies the initial conditions. As described in Fig. 4, At $t = 0$, the exponential term equals one and $\tanh$ is 0, so $\hat{u}(\mathbf{x}, t; \theta) = g(\mathbf{x})$. For $t > 0$ the exponential term decays and the network term grows, allowing the PINN to learn the wavefield without re-learning the initial state. Because the initial conditions are satisfied by construction, we minimize only L_{PDE} .

3.3 Fourier Neural Network

PINNs typically use a standard MLP, which exhibits spectral bias toward low frequencies and hinders the learning of high-frequency wavefield components in heterogeneous media. Neural tangent kernel theory links this to early alignment with smooth NTK eigenmodes (Jacot et al., Wang et al., Rahaman et al.).

To address this, we use Random Fourier Feature mappings (Tancik et al.), which takes the input $\mathbf{x}(t, x, y)$ and maps it into a higher-dimensional space with high-frequency signals before being sent to the MLP using the function:

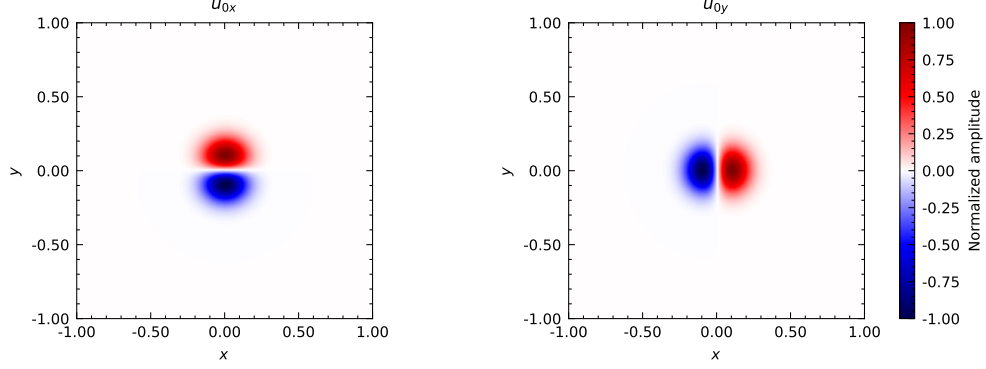


Fig. 3: Initial displacement field conditions used in all experiments. (Left) u_{0x} component; (Right) u_{0y} component of the normalized displacement derived from the second derivative of a Gaussian and y represents space domains in orthogonal directions.

$$\Phi(\mathbf{x}) = \begin{bmatrix} \cos(\mathbf{B}\mathbf{x}) \\ \sin(\mathbf{B}\mathbf{x}) \end{bmatrix}$$

where, $\mathbf{B} \in \mathbb{R}^{m \times d}$ consists of entries sampled from a Gaussian distribution $\mathcal{N}(0, s^2)$, with the standard deviation $s > 0$; m is the number of Fourier modes; and d is the input dimension. In our work, we treat $\mathbf{B}$ as a trainable parameter and optimize it jointly with network parameters. Note that, while previous RFF-PINN studies focused on scalar acoustic wave equations [Sethi et al.], our work extends the approach to the vector-valued elastodynamic system, demonstrating accurate recovery of coupled compressional- and shear-wave fields in strongly heterogeneous media.

3.4 PirateNets

Accurately modelling elastic wave propagation in heterogeneous media requires PINNs with larger and deeper network architectures for providing greater expressibility. However, despite this need for more complex architectures, PINN performance deteriorates as depth increases because standard MLP initialization amplifies vanishing and exploding gradients, destabilizing the optimization of the PDE-residual loss. To overcome these limitations, Wang et al. introduced **PirateNets**, an architecture for deep physics-informed neural networks that facilitates practical training while maintaining computational efficiency by incorporating a coordinate embedding, along with adaptive skip connections. The following sequence of operations defines the architecture.

First, the input coordinates $\mathbf{x}$ are mapped into a higher-dimensional feature space using random Fourier features $\Phi(\mathbf{x})$. Then, this feature vector $\Phi(\mathbf{x})$ is passed through two dense layers to produce gating vectors $\mathbf{U}$ and $\mathbf{V}$:

$$\mathbf{U} = \varphi(W_1 \Phi(\mathbf{x}) + b_1), \quad (12)$$

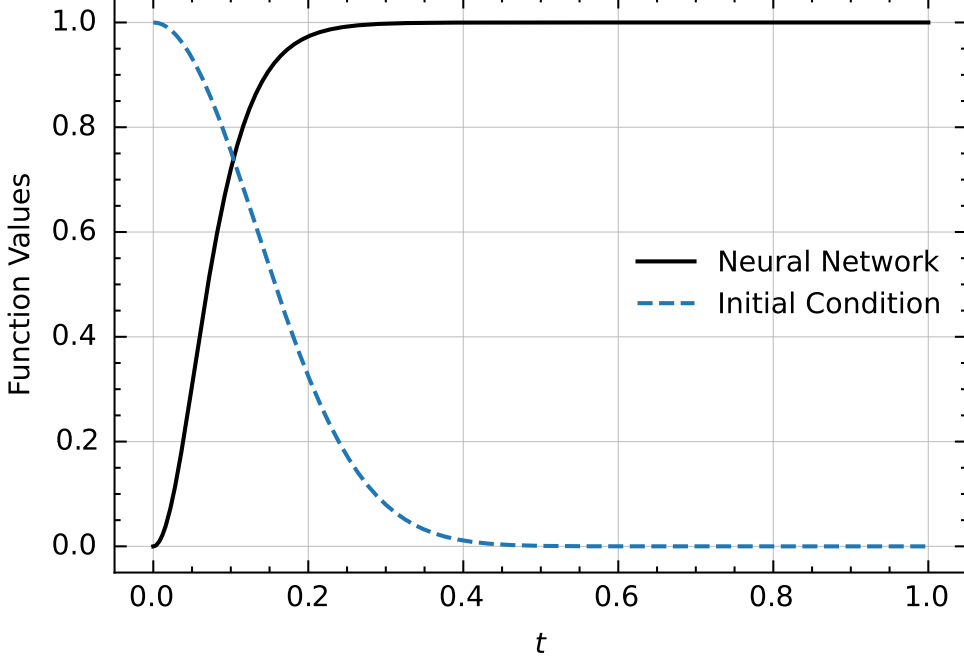


Fig. 4: Temporal evolution of the neural network ansatz enforcing initial conditions. The plot shows how the exponential decay and tanh functions in the ansatz (Equation 11) combine to satisfy $\hat{u}(x, 0; \theta) = g(x)$ at $t = 0$ and allow the network to learn for $t > 0$.

$$\mathbf{V} = \varphi(W_2 \Phi(\mathbf{x}) + b_2), \quad (13)$$

where φ denotes a pointwise activation function. These gating vectors regulate the information flow within the main network, which comprises L sequential residual blocks. The following set of equations defines the forward pass for the l -th block:

$$\mathbf{f}^{(l)} = \varphi(W_1^{(l)} \mathbf{x}^{(l)} + b_1^{(l)}), \quad (14)$$

$$\mathbf{z}_1^{(l)} = \mathbf{f}^{(l)} \odot \mathbf{U} + (1 - \mathbf{f}^{(l)}) \odot \mathbf{V}, \quad (15)$$

$$\mathbf{g}^{(l)} = \varphi(W_2^{(l)} \mathbf{z}_1^{(l)} + b_2^{(l)}), \quad (16)$$

$$\mathbf{z}_2^{(l)} = \mathbf{g}^{(l)} \odot \mathbf{U} + (1 - \mathbf{g}^{(l)}) \odot \mathbf{V}, \quad (17)$$

$$\mathbf{h}^{(l)} = \varphi(W_3^{(l)} \mathbf{z}_2^{(l)} + b_3^{(l)}), \quad (18)$$

$$\mathbf{x}^{(l+1)} = \alpha^{(l)} \mathbf{h}^{(l)} + (1 - \alpha^{(l)}) \mathbf{x}^{(l)}. \quad (19)$$

The skip connection in the final step is adaptively weighted by a **trainable scalar parameter** $\alpha^{(l)}$. To ensure stable training, the weights are initialized using the Glorot

scheme (Glorot and Bengio), and all biases are set to zero. After the final block, the output is produced by a linear layer:

$$\hat{u}(\mathbf{x}; \theta) = W^{(L+1)} \mathbf{x}^{(L)} \quad (20)$$

This adaptive formulation allows PirateNets to overcome the degradation problem and effectively learn multi-scale solutions. Following their work, we initialize each $\alpha^{(l)}$ to zero. This initialization leads to the network starting as a shallow model and effectively deepens itself during training as each $\alpha^{(l)}$ grows. Each block consists of three dense layers, with dual gating operations and the adaptive residual connection.

In their original work [20], the authors propose a physics-informed initialization scheme that fits the final layer using a least-squares solution. However, we found this specific scheme to be incompatible with the hard enforcement of initial conditions used in our approach (Section 3.2). For further details on the PirateNet architecture, we refer the reader to the original paper by Wang et al..

3.5 Training and Optimization

Our goal is to find network parameters θ that minimize the physics loss $J(\theta) = L_{\text{PDE}}(\theta)$:

$$\theta^* = \arg \min_{\theta} J(\theta). \quad (21)$$

Although stochastic and adaptive first-order methods, such as Stochastic Gradient Descent (SGD) and Adam (Robbins and Monro, Goodfellow et al.), are widely used, all experiments in this work are conducted solely using L-BFGS, a quasi-Newton optimizer (Liu and Nocedal). L-BFGS leverages curvature information and is particularly well-suited for our full-batch, smooth, residual-only objective with strict initial conditions. As a member of the quasi-Newton family, BFGS is especially advantageous for navigating the complex and non-convex loss landscapes commonly encountered in PINNs. It iteratively approximates the Hessian matrix of the loss function through sequential gradient evaluations, requiring $O(n^2)$ computational complexity, where n represents the number of model parameters. A key computational benefit of BFGS is its ability to update the Hessian approximation without the need for explicit matrix inversions, which significantly reduces the associated computational costs.

All models are trained using full-batch L-BFGS optimization, employing a strong Wolfe line search procedure. In each experimental setup, two optimizer parameters are independently defined: the learning rate (`lr`) and the memory size (`history_size`), with these parameters reported alongside the results. It is important to note that the line search determines the actual step size, rendering `lr` merely an initial scaling factor. The L-BFGS method operates in full batch mode, wherein a single update utilizes the entire collocation set. This approach generally results in faster convergence compared to mini-batch methods; however, it necessitates a greater allocation of GPU memory.

All experiments were run on a single NVIDIA H100 GPU. Unless otherwise stated, we use a fixed random seed of 42 for all libraries and sampling routines. Collocation points for the physics residual are generated using low-discrepancy Sobol sequences over (t, x, y) , with $N_{\text{PDE}} = 10^5$ points per experiment unless explicitly indicated otherwise. We train in full-batch mode on these N_{PDE} points. Throughout the paper

we use the term *epoch* to denote one optimizer update (i.e., one training iteration). For L-BFGS, an epoch corresponds to a single outer iteration of the optimizer (independent of its internal line-search evaluations). Training curves are therefore reported in *epochs* as defined here.

3.6 Reference Solver

We follow the methodology outlined in [17] to construct our benchmark and generate a high-fidelity reference using DEVITO [31]. DEVITO is a finite-difference solver that integrates symbolic algebra with optimized stencil generation. Specifically, we solve the displacement-form elastic wave equation on a 512×512 grid with a time step of $\Delta t = 10^{-3}$ s, employing a second-order scheme in time and a fourth-order scheme in space.

Since the output from DEVITO is numerical rather than analytic, the reported L_2 errors reflect both the errors from the PINN and the discretization error of the reference solution. The small variance observed across different time steps arises from this numerical uncertainty and decreases with further grid refinement. We use a numerical benchmark because our goal is to evaluate PINNs in heterogeneous media with sharp interfaces—such as reflections, refractions, and mode conversions—where analytic solutions are typically not available.

Consistent with the findings in [17], a small offset may appear at $t = 0$, even with hard initial conditions. This occurs because a second-order time discretization requires data from two different time levels. Therefore, the DEVITO solution should be viewed as a high-fidelity approximation. The actual PINN error is likely slightly lower than the reported value, but without an analytic solution, we cannot quantify this margin precisely.

3.7 Evaluation Metrics

To quantitatively assess the performance of the PINN, we use two independent metrics. At each time step t_i , we compute these metrics to compare the PINN’s predicted solution (U_{NN}) against the reference solution (U_{GT}).

3.7.1 L_2 Relative Error

The relative L_2 error provides a normalized measure of the overall difference between the reference and predicted wavefields at a specific moment in time. The average of this error over the entire simulation is used as our metric for overall model accuracy.

The relative L_2 Error is given by:

$$L_2 \text{ Error} = \frac{\|U_{\text{GT}} - U_{\text{NN}}\|_2}{\|U_{\text{GT}}\|_2}, \quad (22)$$

where $\|\cdot\|_2$ denotes the L^2 norm over the spatial domain. Here, U_{GT} refers to the reference solution generated using DEVITO and U_{NN} denotes the PINN solution.

3.7.2 Normalized Energy Difference

To complement the L_2 error, we also adopt the Normalized Energy Difference (Sethi et al.) to quantify the difference between PINN solutions and those from traditional numerical methods. This metric is particularly well-suited for wave propagation problems as it evaluates the mismatch in the wavefield’s energy. It is defined as:

$$E(t) = \frac{\sum_{N_x} \sum_{N_y} (U_{\text{NN}}(t, x, y) - U_{\text{GT}}(t, x, y))^2}{\sum_{N_x} \sum_{N_y} U_{\text{GT}}^2(t, x, y)}$$

where $U_{\text{NN}}(t, x, y)$ is the solution from the PINN, and $U_{\text{GT}}(t, x, y)$ is the reference solution from the numerical method.

As justified in [22], this metric offers distinct advantages. Using a squared difference in the numerator makes the metric highly sensitive to large errors in wave amplitude, which is a desirable property for assessing wavefield accuracy. Further, normalizing by the total energy of the reference solution (the denominator) renders the metric relative and dimensionless. This is crucial for making fair comparisons of model performance across different time steps, where the absolute magnitude of the wavefield naturally fluctuates. This approach provides a robust method for error quantification that is consistent with the relative L_2 Error defined previously.

4 Benchmarks

This section details the results of our comparative analysis of three distinct PINN architectures namely, the vanilla PINN, the Fourier Feature PINN, and the PirateNet, in the context of solving the elastic wave equation. We present results for three synthetic test cases of increasing complexity: (i) a homogeneous model, (ii) a single-layered model, and (iii) a multi-layered model.

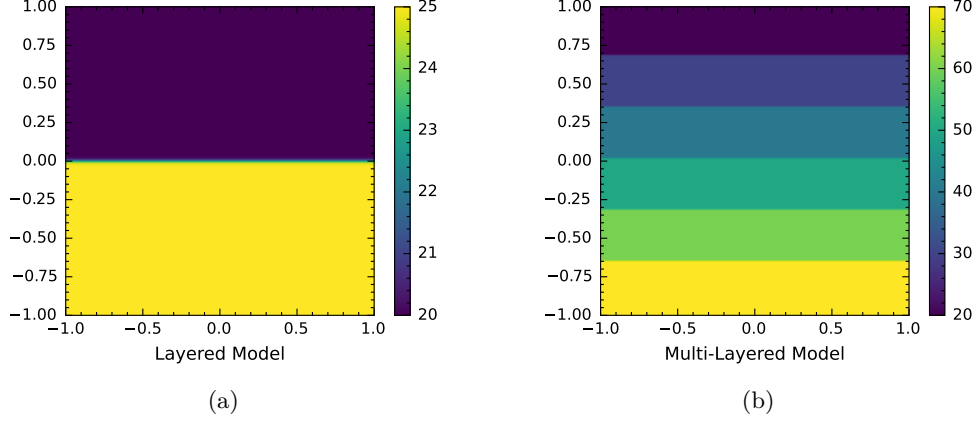


Fig. 5: Heterogeneous parameter models for Lamé parameter λ . (a) Two-layered model with an interface at $y = 0$; (b) Multi-layered model with six equal-thickness layers. The density $\rho = 100$ is used in all experiments.

4.1 Homogeneous Model

As an initial baseline, we consider a 2-D homogeneous elastic medium with constant Lamé’s parameters, $\lambda = 20$, and $\mu = 30$ and constant density $\rho = 100$. These values yield analytical P- and S-wave speeds of $V_p = 0.89$ and $V_s = 0.54$ via (Eq. (4)).

We generate 10^5 collocation points using Sobol sequences within the range $\mathbf{x}(t, x, y) \in [0, 1]s \times [-1, 1]^2$. To ensure a fair comparison, we utilize the same set of points for all model variants. We implement the initial conditions as outlined in section 3.2 and employ the L-BFGS method for training, setting the learning rate (lr) to 1.0 and maintaining a history size of 100. This configuration is consistently applied across all experiments conducted on the homogeneous model.

The vanilla PINN comprises five hidden layers with 128 units each, and hyperbolic tangent as the activation function. Vanilla PINN takes 20000 epochs to converge, giving an average L_2 error of 5.38%.

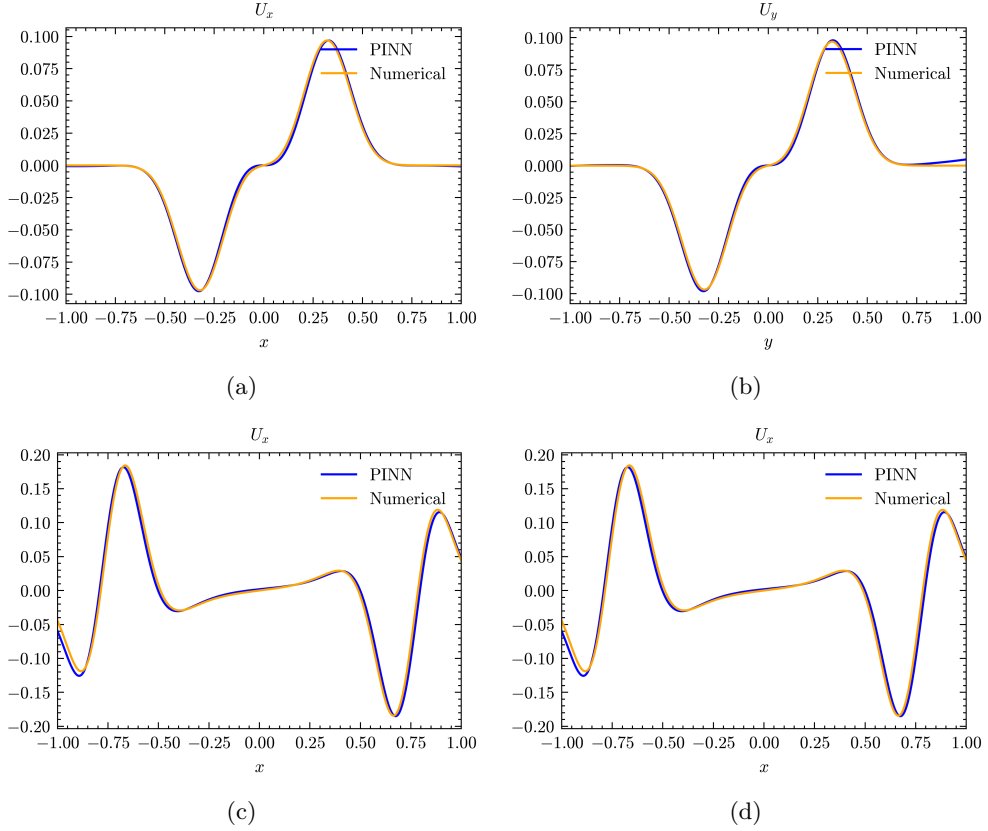


Fig. 6: Displacement field traces for the homogeneous model comparing PirateNet with numerical reference. (a,c) Horizontal displacement u_x along $y = 0$; (b,d) Vertical displacement u_y along $x = 0$, shown at $t = 0.5$ s (a,b) and $t = 0.9$ s (c,d).

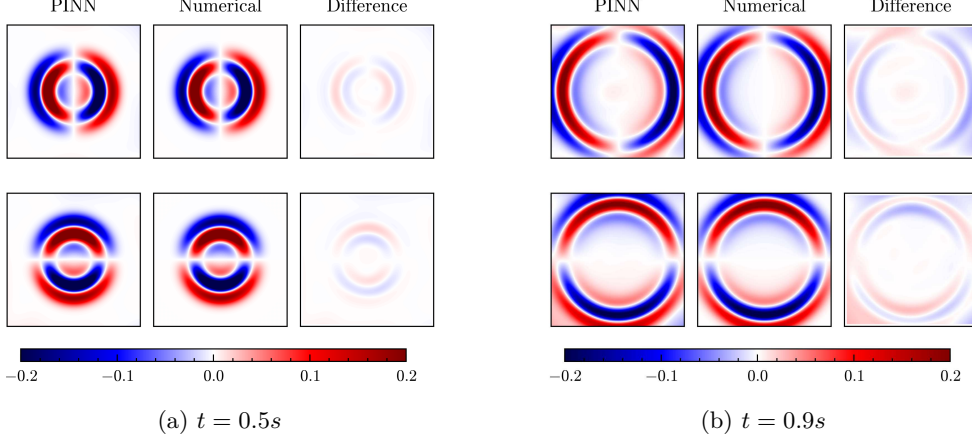


Fig. 7: Wavefield snapshots for the homogeneous velocity model. The left and center columns show the PirateNet, PINN and numerical solutions, respectively, with the right column showing their differences. The first row displays the U_x field, and the second row the U_y field. Subfigures (a) and (b) show results at 0.50 s and 0.99 s.

Next, our Fourier-feature PINN used five layers with 128 units, a trainable random Fourier feature layer with a size $m = 128$, and a standard deviation $s = 1$. It takes 20,000 epochs to converge, giving a final average L_2 error of 5.20%.

The PirateNet model used 2 blocks with 3 dense layers in each, and 64 units. The trainable random Fourier layer is of size 64 and has a standard deviation of $s = 2$. The traces shown in Fig. 6 are from the PirateNet model.

In Fig. 7 the left column shows the PINN prediction, the centre column shows the numerical solution, and the right column shows the pointwise difference. The first row displays the U_x displacement field, and the second row the U_y field. Fig. 7 shows the general agreement between the PINN and FDM solutions.

Fig. 8 (a) demonstrates that the vanilla PINN, Fourier-feature PINN, and PirateNet achieve practically identical final accuracies, with the normalized energy misfit being highest for PirateNet. The Fourier-feature PINN attains the least energy difference, but the margin is negligible for this homogeneous benchmark.

A quantitative summary of the final accuracies is provided in Table 1, confirming that all three architectures achieve a comparable relative L_2 error.

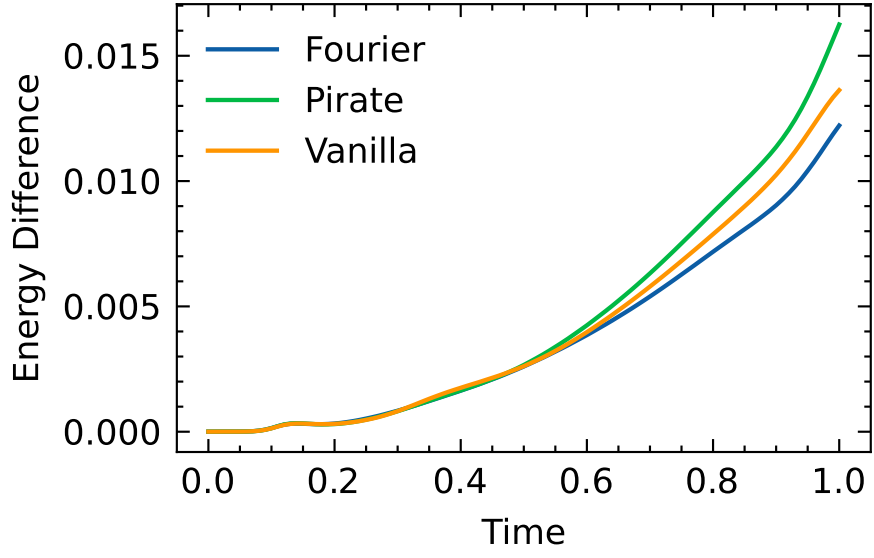
The training loss depicted in Fig. 8 (b) highlights notable differences in efficiency among the networks. PirateNet achieves a loss plateau in approximately 5000 epochs, the Fourier-feature PINN in about 6000 epochs, and the vanilla PINN in nearly 20,000 epochs. This indicates that PirateNet converges approximately four times faster in terms of optimization steps while ultimately matching the accuracy of the other two, showcasing a substantial enhancement in training efficiency.

In summary, Fig. 8 illustrates that while all three networks achieve similar accuracy, PirateNet accomplishes this with significantly fewer optimization steps. This makes PirateNet a more attractive option for larger or more complex problems. The

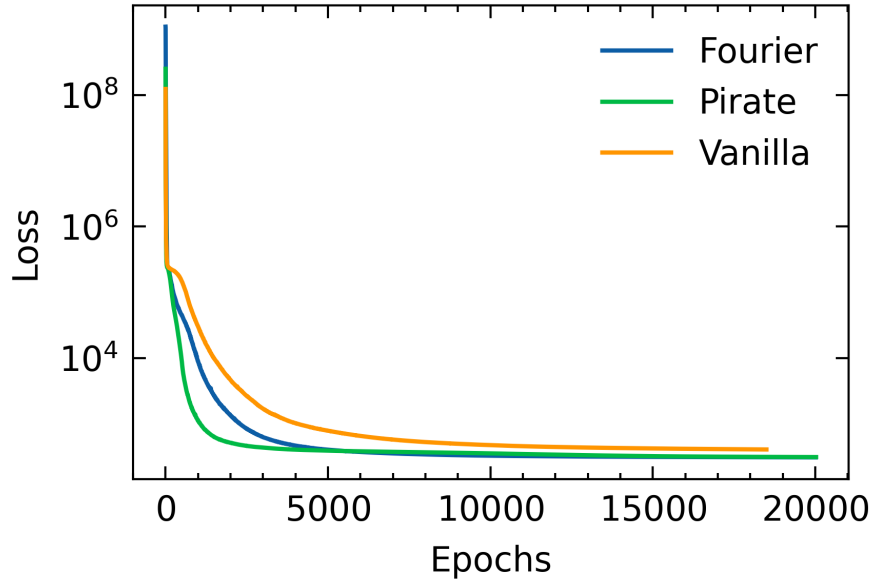
faster convergence of PirateNet proves especially advantageous in the heterogeneous and larger-scale experiments, as described in [Section 4.2](#) and [Section 4.3](#).

Table 1: Performance comparison of vanilla PINN, Fourier-feature PINN, and PirateNet variants.

Configuration	Relative L_2 Error (%)
Vanilla PINN	5.38
Fourier-feature PINN	5.21
PirateNet	5.58



(a) Time(s)



(b)

Fig. 8: Performance comparison for the homogeneous model. (a) Normalized energy difference over time for Vanilla PINN, Fourier-feature PINN, and PirateNet. (b) Training loss convergence showing optimization epochs versus residual loss.

4.2 Two Layered Model

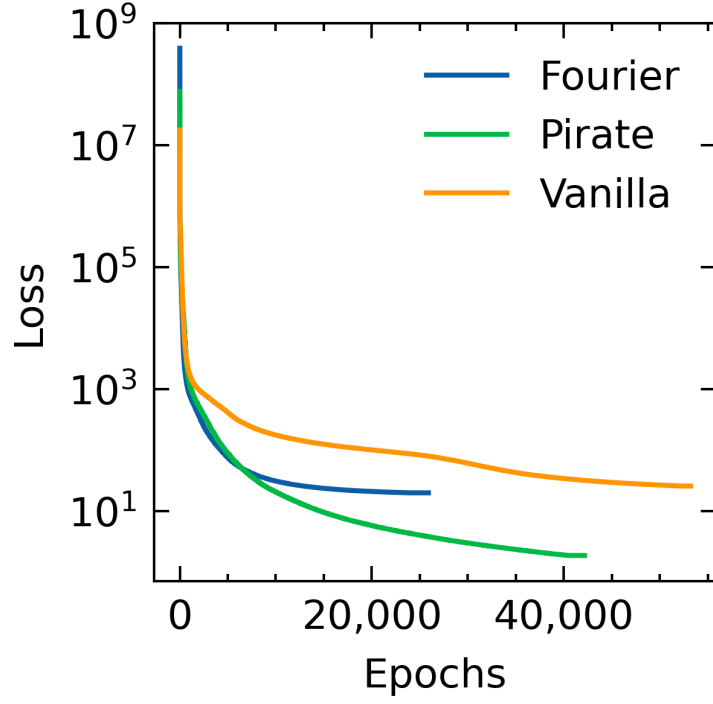
In the second numerical experiment, we evaluated three network architectures using a heterogeneous two-layer elastic model, as illustrated in Fig. 5a. The computational domain was defined as $[-1, 1]^2 \times [0, 1]$ s, with an interface located at $y = 0$. Above this interface, the material properties were specified as $\rho = 100$, $\lambda = 20$, and $\mu = 30$. Below the interface, the properties were set to $\rho = 100$, $\lambda = 25$, and $\mu = 35$. To manage sharp gradients, the transition in Lamé parameters across the interface was smoothed using a hyperbolic tangent function.

The same initial conditions as described in Section 3.2 were utilized, and the L-BFGS optimization method was used, setting the learning rate (lr) to 2.0 and maintaining a history size of 100. This configuration was consistently applied across all experiments conducted on this model, along with $N_{PDE} = 10^5$ spatio-temporal training points.

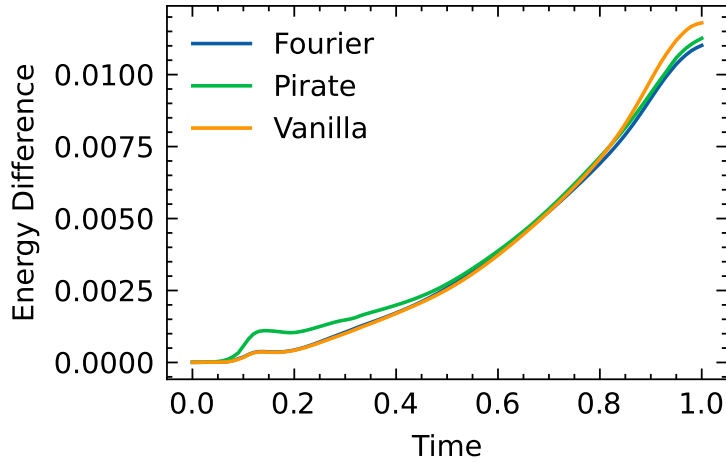
The vanilla-PINN architecture consists of five hidden layers, each containing 128 units with *tanh* activation. As shown in Fig. 9a, the training loss for the vanilla-PINN stabilized at approximately 10^2 after 50000 iterations, resulting in a relative L_2 error of 5.29%. It exhibited a lower energy difference in the early time steps; however, this difference increased in later time predictions compared to the other models, as illustrated in Fig. 9b. Thus, the vanilla PINN effectively learned a significant portion of the wavefield. However, the increasing difference at later time steps indicates a failure to fully capture the reflections and low-scale features generated by the interface. This limitation also contributes to why the loss remains above the order of 10^2 . It indicates that the network resolved earlier wavefronts tied to initial conditions, but fails to fully capture higher-frequency reflections and mode conversions generated at the material interface.

Next, we train a Fourier-feature PINN consisting of five hidden layers, each containing 128 units, with a *tanh* activation function, along with a Fourier feature layer characterized by $m = 128$ and a standard deviation $s = 1.0$. As illustrated in Fig. 9a, the Fourier-feature PINN achieves convergence to an order of 10^1 in under 30,000 epochs. Notably, it not only attains a lower order of loss but does so in fewer training epochs. The reduced residual loss indicates that the model effectively captures the high-frequency component of the wavefield associated with the interface. Furthermore, the energy difference for this model is considerably lower than that observed for the vanilla PINN, as depicted in Fig. 9b. The final relative L_2 error achieved is 5.25%, which is an improvement over the Vanilla PINN model. This observation highlights that Fourier feature mapping not only enhances the accuracy of the solution but also accelerates the convergence of the network.

Lastly, we used the PirateNet architecture, consisting of two residual blocks, each containing 64 hidden units. This results in a total of six layers and utilizes a Fourier feature mapping with $N = 64$ and a standard deviation $s = 1.0$. The training loss for PirateNet achieves $\mathcal{O}(10^2)$ in under 10,000 epochs, outperforming the other models. Notably, it converges to a loss value that is two orders of magnitude lower than that of the Vanilla model and one order of magnitude lower than that of the Fourier



(a)



(b)

Fig. 9: Performance metrics for the two-layered model. (a) Training loss convergence for Vanilla PINN, Fourier-feature PINN, and PirateNet showing epochs versus residual loss. (b) Normalized energy difference over time for all three architectures.

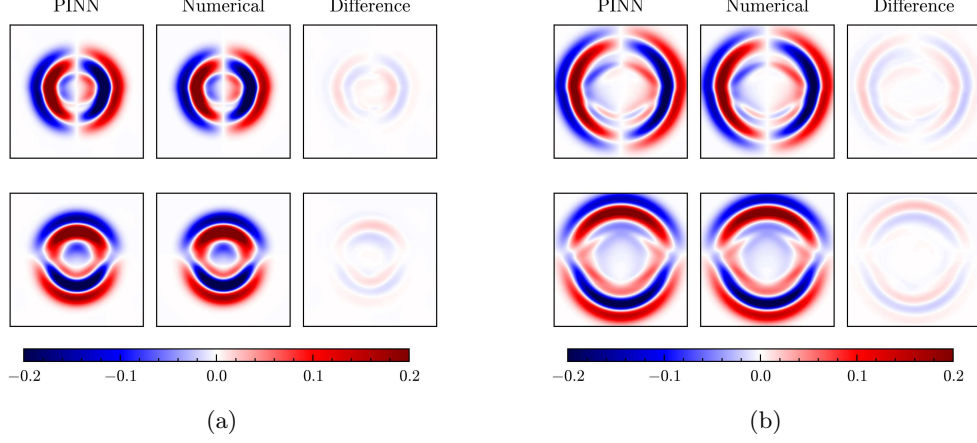


Fig. 10: Wavefield snapshots for the two-layered velocity model comparing PirateNet to the numerical reference. (a) Results at $t = 0.20$ s, (b) at $t = 0.50$ s. Each panel: left—PirateNet, center—numerical, right—pointwise difference. Top row: u_x displacement; bottom row: u_y displacement.

model. Although we observed a slightly increased energy misfit for PirateNet at the initial time steps—attributed to the strict enforcement of initial conditions—this misfit diminishes after $t = 0.5$ s and ultimately converges to the same value as the Fourier-feature PINN. The final relative L_2 error achieved was 5.60%.

The final performance metrics for all three architectures are consolidated in [Section 4.2](#), which shows that all models achieved a relative L_2 error of approximately 5.5%.

In [Fig. 10](#), the left column illustrates the PINN prediction, the center column presents the numerical solution, and the right column depicts the pointwise difference. The first row showcases the U_x displacement field, while the second row displays the U_y field. We observe a strong and nearly indistinguishable agreement between the displacement fields. [Fig. 11](#) illustrates the traces of the wavefield obtained through PirateNet in comparison to the numerical solution. This demonstrates that PirateNet effectively captures not only the large-scale features of the wave fields but also the fine-scale, high-frequency content associated with reflections and mode conversions. The residual skip connections help maintain gradient flow during late-stage training. At the same time, the Fourier features introduce the appropriate basis functions, working together harmoniously.

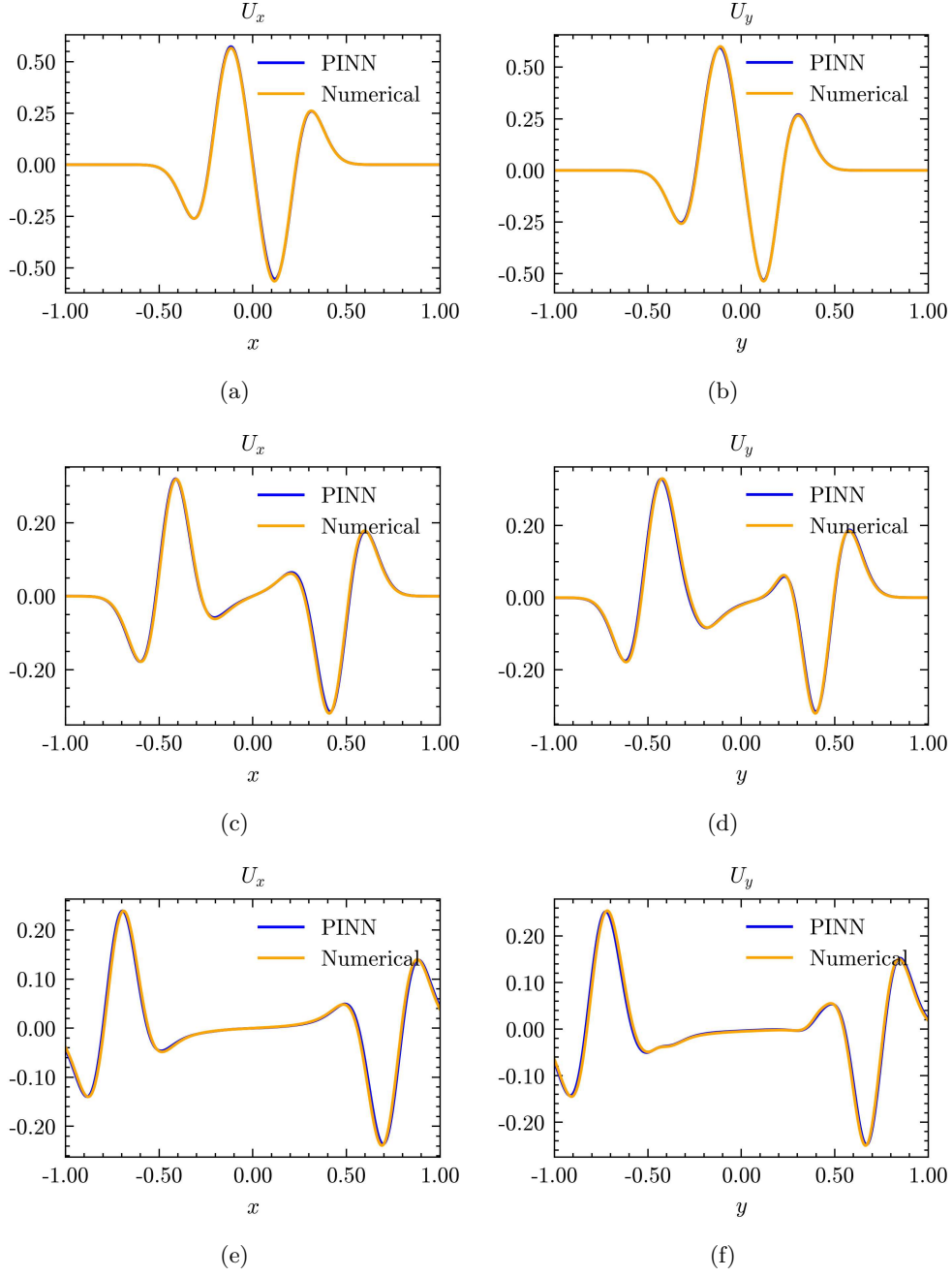


Fig. 11: Displacement field traces for the two-layered model comparing PirateNet with numerical reference. (a,b) $t = 0.2$ s; (c,d) $t = 0.5$ s; (e,f) $t = 0.8$ s. Left column: u_x ; right column: u_y along centerlines.

Configuration	Relative L_2 Error (%)
Vanilla PINN	5.29
Fourier-feature PINN	5.25
PirateNet	5.60

4.3 Multi-layered Model

In this section, we extend our investigation to a highly complex multi-layered medium exhibiting distinct discontinuities in material properties, as illustrated in Figure 5(b). The model has six equal-thickness layers. The computational domain is defined as $[-1.5, 1.5]^2 \times [0, 1]$ s. To address sharp gradients, the transition in Lamé parameters across interfaces is smoothed with a hyperbolic tangent function. Waves propagating through layered interfaces give rise to multiple reflections, P-S conversions, and interference, making it the most challenging experiment in this paper.

The same initial conditions as described in Section 3.2 were employed, utilizing the L-BFGS optimization method, with a learning rate of 2.0 and a history size of 100. This configuration was consistently applied across all experiments conducted on this model, with $N_{PDE} = 10^5$ spatio-temporal training points.

Given the increased complexity of this model, we employ a curriculum learning strategy to stabilize the optimization process, enabling the network to first learn the simpler, early-time wave propagation before being tasked with fitting the complex, high-frequency reflections and mode conversions that emerge at later times. The temporal intervals are incrementally expanded, with the network first trained on $[0, 0.3]$ s, then $[0, 0.5]$ s, $[0, 0.8]$ s, and finally $[0, 1.0]$ s, warm-starting each stage from the previous weights and resampling the collocation set.

We first evaluate the vanilla PINN, consisting of five hidden layers with 128 neurons each. This model failed to capture the whole wavefield, primarily fitting low-frequency components while failing to resolve sharp features near material interfaces and was ineffective in learning the internal reflections generated by the layers.

These behaviors are illustrated in the displacement field snapshots and trace plots shown in Fig. 12 and Fig. 13. In the top row (a–b) of Fig. 12, the displacement snapshots highlight this issue: the difference column reveals ring-like artifacts aligned with the interfaces at both $t = 0.50$ s and $t = 0.90$ s. Corresponding centerline traces in Fig. 13 (vanilla, blue versus reference, orange) show phase lead/lag at major arrivals and amplitude suppression near sharp features. We observed relative L_2 errors of 38% and 39% at $t = 0.5$ s and $t = 0.9$ s, respectively. The vanilla PINN was unable to capture the sharp feature, effectively fitting only the low spatial frequency wavefield components. However, the resulting smooth wavefields failed to capture localized behaviour.

Subsequently, we train a Fourier-feature PINN with five hidden layers, 128 units, trainable Fourier features with $m = 64$. The Fourier-feature PINN also fails to converge in this multi-layered setting. The wavefields generated by Fourier-feature PINN can be observed in Fig. 12c and Fig. 12d.

These limitations are evident in the displacement snapshots and trace plots shown in Figures 12 and 13. Both vanilla and Fourier-feature PINNs exhibit large discrepancies compared to the reference solution, particularly near high-contrast interfaces and at later time steps.

In contrast, a deep residual-adaptive backbone network allocates capacity differently during optimization. Hence, we evaluate the PirateNet architecture, configured with three blocks equivalent to nine hidden layers, each with 128 neurons. PirateNet

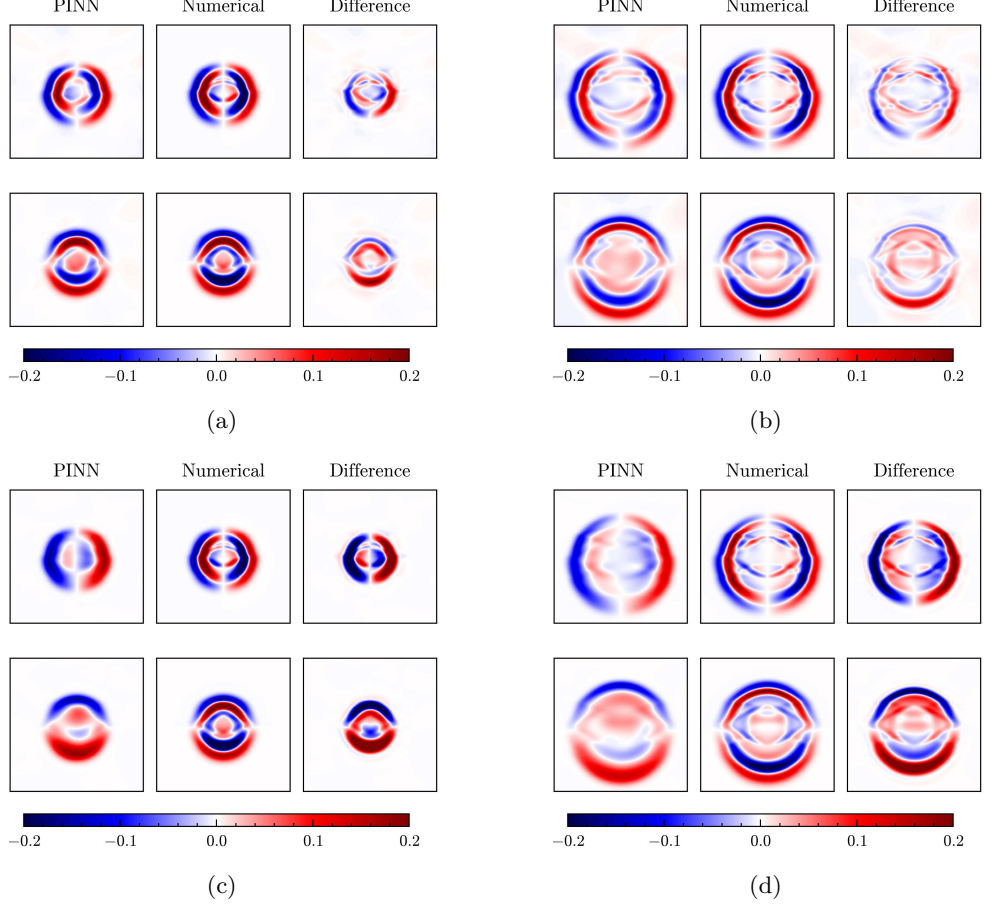


Fig. 12: Wavefield snapshots for the multi-layered model highlighting failures of standard PINN and Fourier-feature PINN. (a,b) Vanilla PINN at $t = 0.50$ s and $t = 0.90$ s. (c,d) Fourier-feature PINN at $t = 0.50$ s and $t = 0.90$ s. Each: left—PINN, center—numerical, right—pointwise difference (u_x).

was able to capture the internal reflection generated by the layers and a sharp wavefield, as demonstrated in Figure 14. Displacement traces in Figure 15 further confirm that PirateNet closely follows the reference solution, with only minor deviations that can be attributed to numerical and discretization effects. The relative L_2 error for the snapshots shown in Fig. 14a is approximately 15% at $t=0.50$ s and 14% at $t=0.90$ s. We observe good phase agreement between the arrivals originating from sharp gradients; however, there are minor differences in the amplitude of the displacement wavefield. The use of PirateNet improved the overall results compared to vanilla pinn and Fourier-feature PINN.

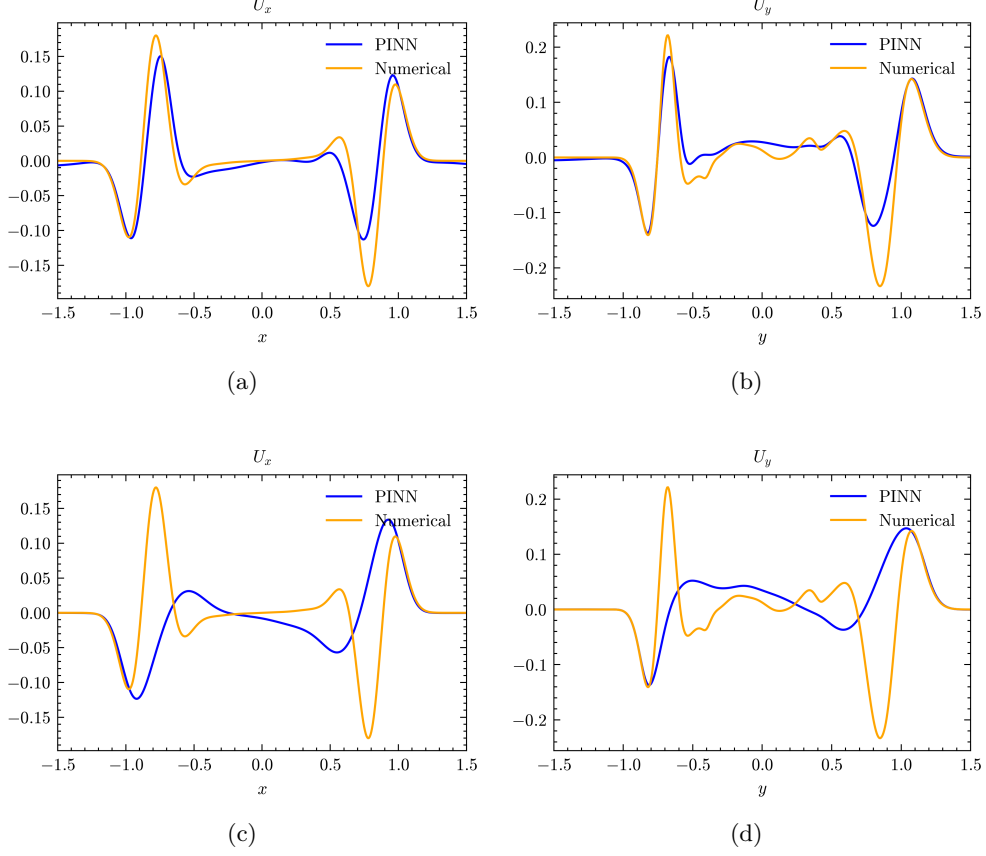


Fig. 13: Wavefield differences for Vanilla PINN and Fourier Feature PINN at $t = 0.9$ s. The first row (a) and (b) shows the differences from Vanilla PINN, while the second row (c) and (d) shows differences from Fourier Feature PINN.

A quantitative summary of errors is provided in [Table 2](#), which shows that PirateNet achieves a substantially lower relative L_2 error compared to either vanilla or Fourier-feature PINNs.

In summary, only the residual-adaptive PirateNet architecture is able to robustly model both large-scale and high-frequency features in multi-layered elastic media under the described training setup. Vanilla and Fourier-feature PINNs fail to converge or miss essential physics, leading to significantly higher errors.

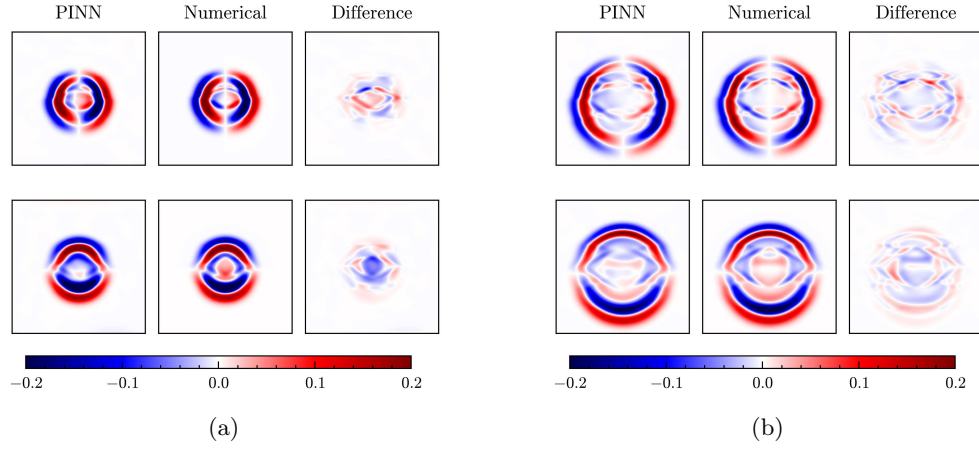


Fig. 14: Snapshots for the multi-layered velocity model using PirateNet. (a) $t = 0.50$ s, (b) $t = 0.90$ s. Left: PirateNet prediction; center: numerical reference; right: pointwise difference. Upper row: u_x ; lower row: u_y .

Table 2: Comparison of Vanilla PINN, Fourier-feature PINN and PirateNet on Multi Layered Model.

Configuration	Avg. Relative L_2 Error (%)
Vanilla PINN	34.52
Fourier-feature PINN	58.51
PirateNet	10.61

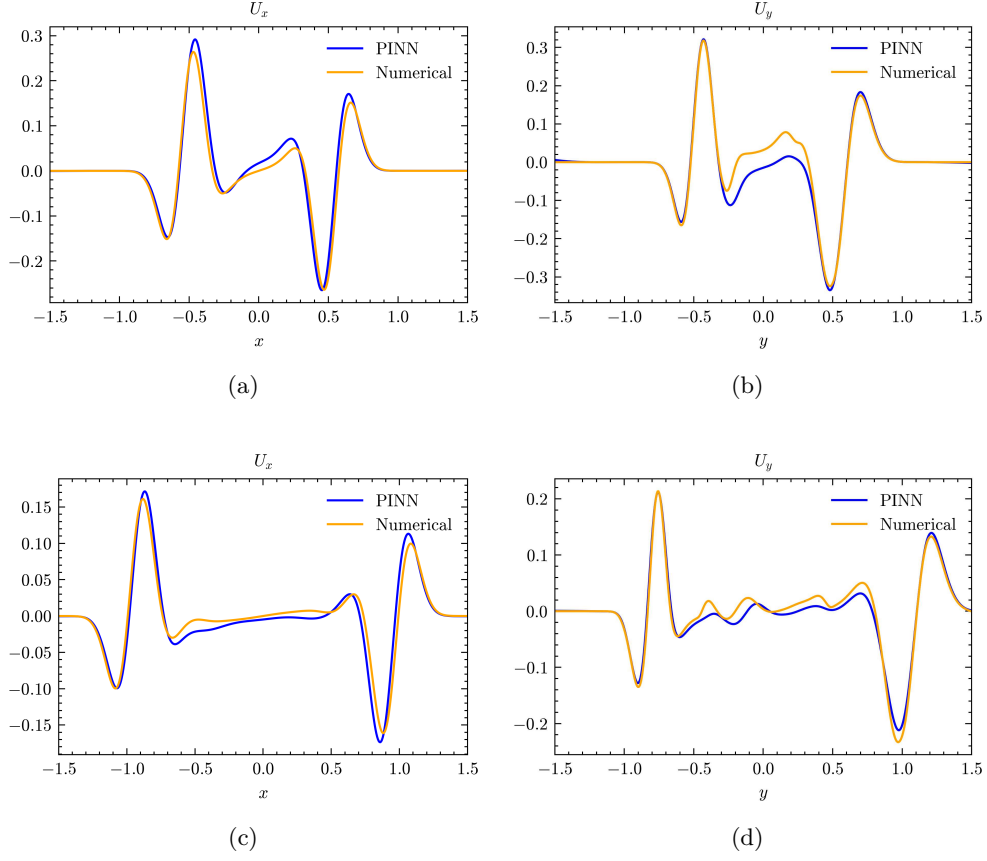


Fig. 15: Displacement field traces for the multi-layered model comparing PirateNet with numerical reference . (a, b) Results at $t = 0.5$ s; (c, d) Results at $t = 0.9$ s. Left column: horizontal displacement u_x ; Right column: vertical displacement u_y along centerlines.

5 Discussion

This work demonstrates that physics-informed neural networks (PINNs) can effectively learn elastodynamic wavefields in both homogeneous and heterogeneous media. However, their performance significantly depends on architectural choices and embedding strategies. In homogeneous cases, all tested models—vanilla PINNs, Fourier-feature PINNs, and PirateNets—achieved comparable accuracy, suggesting that the PDE residual alone is sufficient when the solution remains spectrally simple. However, when heterogeneity was introduced, Fourier embeddings improved performance by reducing low-frequency bias. In the most complex six-layer configuration, only PirateNet converged reliably and captured the high-frequency content necessary for internal reflections and P–S conversions. PirateNet reduced the relative L_2 error to 10.6%, compared with 34.5% for the vanilla PINN and 58.5% for the Fourier-feature PINN. This order-of-magnitude improvement highlights the advantage of adaptive skip connections for capturing high-frequency wavefields.

These findings have several implications for computational elastodynamics. Firstly, they clarify the role of architectural bias: spectral embeddings are adequate for moderately heterogeneous settings, while residual-adaptive skip connections are essential in strongly stratified media and offer stable training, outperforming traditional ResNet architectures. This provides practical guidance for researchers and practitioners in selecting between lightweight and more sophisticated PINN architectures. Secondly, the differentiable nature of PINN surrogates makes them particularly well-suited for inverse problems, such as full-waveform inversion (FWI) and non-destructive testing (NDT), where sensitivity to material parameters and geometry is crucial. By delivering continuous spatio-temporal solutions, PINNs extend beyond traditional finite-difference solvers, enabling gradient-based optimization and integration with machine learning pipelines. Notably, PirateNets’ ability to learn sharp features makes them suitable for reconstructing sharp FWI images in the monitoring of solids.

Previous studies on acoustic wave equations have reported similar patterns, where spectral bias limited the recovery of high-frequency modes and architectural modifications enhanced convergence. Our results extend these insights to elastodynamics, illustrating that the interaction between Fourier features and residual-adaptive designs determines whether PINNs can effectively model layered elastic media. Notably, the failure of both vanilla and Fourier-feature PINNs in the multi-layered context aligns with Staub’s observation that simple architectures struggle in strongly discontinuous settings, while our findings position PirateNet as a viable alternative. The vanilla PINNs utilized by [Rasht-Behesht et al.](#) had difficulty identifying all internally reflected phases at abrupt material boundaries, leading to less accurate inversion outcomes.

Despite these advantages, several limitations must be acknowledged. First, PINNs require greater computational resources than conventional solvers, and they still do not achieve the accuracy levels of spectral solvers. Additionally, our source treatment relied exclusively on initial conditions and avoided explicit forcing terms; efforts to introduce singular sources resulted in instabilities, indicating a need for formulations based on scattered fields or distributed sources. Further, our collocation sampling was uniform and not tailored to interfaces, which likely led to an under-representation

of small-amplitude scattered arrivals. The requirement for full batch training with L-BFGS is also not practical for large-scale problems due to limited GPU availability.

Building on these findings, several promising research directions emerge. From a feature design perspective, replacing the Fourier embeddings with wavelets or Gabor functions could combine frequency representation with spatial locality, thereby enhancing the overall representation. Architecturally, integrating adaptive blocks within encoder-decoder backbones or domain-decomposition frameworks could further localize high frequencies while enhancing scalability. On the optimization front, a recent work by [Wang et al.](#) has demonstrated that the SOAP optimizer can aid in solving the 3D turbulence problem; however, its performance in solving wave dynamics remains unclear. In terms of source modelling, scattered-field formulations could help regularize singularities and enhance stability. Finally, evaluation practices should extend beyond relative L_2 error to include metrics such as time-to-tolerance, energy misfit decay, and robustness under noise, all of which are crucial for real-world structural health monitoring (SHM) and non-destructive testing applications. [Moreh et al.](#) also used deep learning to solve the problem of damage identification in structures using deep learning.

In summary, this study advances the use of PINNs for elastodynamics from a proof-of-concept stage towards practical surrogates for wavefield modelling in heterogeneous media. By clarifying the scenarios where Fourier embeddings are sufficient and those where residual-adaptive designs are necessary, it establishes a foundation for robust, differentiable surrogates, paving the way for their application in gradient-based inversion tasks, such as full-wave inversion in medical imaging, geophysics, structural health monitoring, and non-destructive evaluation.

Acknowledgements. This research was supported in part through high-performance computing resources available at the Kiel University Computing Centre.

Declarations

- **Funding:** This research was supported by the Central Innovation Programme for SMEs (ZIM) under the project StructMonitor TP1: Entwicklung einer wellenbasierten Schadensdetektion (Project No. FKZ-KK5217903RL4). Open Access funding enabled and organized by Projekt DEAL.
- **Competing interests:** The authors declare no competing interests.
- **Ethics approval and consent to participate:** Not applicable.
- **Consent for publication:** Not applicable.
- **Materials availability:** Not applicable.
- **Code and Data availability:** The code will be made available upon publication at <http://github.com/hannanmustajab/elastodynamics-pinn>.
- **Author contribution:** AHM developed the code and performed the experiments. AHM and ARK wrote the manuscript. ZHR, and FW discussed the results and provided feedback and comments on the manuscript.

References

- [1] Alterman, Z., Karal, F.C.: Propagation of elastic waves in a layered media by finite difference methods. *Bulletin of the Seismological Society of America* **58**(1), 367–392 (1968)
- [2] Komatitsch, D., Vilotte, J.-P.: The spectral element method: an efficient tool to simulate the seismic response of 2d and 3d geological structures. *Bulletin of the Seismological Society of America* **88**(2), 368–394 (1998)
- [3] Virieux, J.: P-sv wave propagation in heterogeneous media: Velocity-stress finite-difference method. *Geophysics* **51**(4), 889–901 (1986)
- [4] Raissi, M., Perdikaris, P., Karniadakis, G.E.: Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics* **378**, 686–707 (2019) <https://doi.org/10.1016/j.jcp.2018.10.045>
- [5] Yang, L., Meng, X., Karniadakis, G.E.: B-pinns: Bayesian physics-informed neural networks for forward and inverse pde problems with noisy data. *Journal of Computational Physics* **425**, 109913 (2021)
- [6] Sarma, A.K., Roy, S., Annavarapu, C., Roy, P., Jagannathan, S.: Interface pinns (i-pinns): A physics-informed neural networks framework for interface problems. *Computer Methods in Applied Mechanics and Engineering* **429**, 117135 (2024)
- [7] Roy, S.R., Chandrasekhar Annavarapu, C.A., Pratanu Roy, P.R., Antareep Kumar Sarma, A.K.S.: Adaptive interface-pinns (adai-pinns): An efficient

- physics-informed neural networks framework for interface problems. *Communications in Computational Physics* **37**(3), 603–622 (2025) <https://doi.org/10.4208/cicp.oa-2024-0131>
- [8] Moseley, B., Markham, A., Nissen-Meyer, T.: Finite basis physics-informed neural networks (fbpinns): a scalable domain decomposition approach for solving differential equations. *Advances in Computational Mathematics* **49**(4), 62 (2023)
 - [9] Yu, J., Lu, L., Meng, X., Karniadakis, G.E.: Gradient-enhanced physics-informed neural networks for forward and inverse pde problems. *Computer Methods in Applied Mechanics and Engineering* **393**, 114823 (2022)
 - [10] Xie, G., Fu, B., Li, H., Du, W., Zhong, Y., Wang, L., Geng, H., Zhang, J., Si, L.: A gradient-enhanced physics-informed neural networks method for the wave equation. *Engineering Analysis with Boundary Elements* **166**, 105802 (2024)
 - [11] Rasht-Behesht, M., Huber, C., Shukla, K., Karniadakis, G.E.: Physics-informed neural networks (pinns) for wave propagation and full waveform inversions. *Journal of Geophysical Research: Solid Earth* **127**(5), 2021–023120 (2022)
 - [12] Mustajab, A.H., Lyu, H., Rizvi, Z., Wuttke, F.: Physics-informed neural networks for high-frequency and multi-scale problems using transfer learning. *Applied Sciences* **14**(8), 3204 (2024)
 - [13] Alkhadhr, S., Almekkawy, M.: Wave equation modeling via physics-informed neural networks: Models of soft and hard constraints for initial and boundary conditions. *Sensors* **23**(5), 2792 (2023)
 - [14] Herrmann, L., Bürchner, T., Dietrich, F., Kollmannsberger, S.: On the use of neural networks for full waveform inversion. *Computer Methods in Applied Mechanics and Engineering* **415**, 116278 (2023)
 - [15] Rao, C., Sun, H., Liu, Y.: Physics-informed deep learning for computational elastodynamics without labeled data. *Journal of Engineering Mechanics* **147**(8), 04021043 (2021)
 - [16] Liang, R., Liu, W., Xu, L., Qu, X., Kaewunruen, S.: Solving elastodynamics via physics-informed neural network frequency domain method. *International Journal of Mechanical Sciences* **258**, 108575 (2023)
 - [17] Staub, D.: Solving the elastic wave equation with physics-informed neural networks: A robust and critical assessment. Master thesis, ETH Zurich, Zurich (2024-03). <https://doi.org/10.3929/ethz-b-000668359>
 - [18] Mu, X., Cheng, S., Alkhalifah, T.: SeparationPINN: Physics-Informed Neural Networks for Seismic P- and S-Wave Mode Separation (2025). <https://arxiv.org/abs/2504.07544>

- [19] Tancik, M., Srinivasan, P., Mildenhall, B., Fridovich-Keil, S., Raghavan, N., Singhal, U., Ramamoorthi, R., Barron, J., Ng, R.: Fourier features let networks learn high frequency functions in low dimensional domains. *Advances in neural information processing systems* **33**, 7537–7547 (2020)
- [20] Wang, S., Li, B., Chen, Y., Perdikaris, P.: Piratenets: Physics-informed deep learning with residual adaptive networks. *Journal of Machine Learning Research* **25**(402), 1–51 (2024)
- [21] Meng, W.J., Fu, L.Y.: Seismic wavefield simulation by a modified finite element method with a perfectly matched layer absorbing boundary. *Journal of Geophysics and Engineering* **14**(4), 852–864 (2017) <https://doi.org/10.1088/1742-2140/aa6b31>
- [22] Sethi, H., Pan, D., Dimitrov, P., Shragge, J., Roth, G., Hester, K.: hard enforcement of physics-informed neural network solutions of acoustic wave propagation. *Computational Geosciences* **27**, 737–751 (2023) <https://doi.org/10.1007/s10596-023-10232-3>
- [23] Aki, K., Richards, P.G.: *Quantitative Seismology*, 2nd edn. University Science Books, Sausalito, California (2002)
- [24] Jacot, A., Gabriel, F., Hongler, C.: Neural tangent kernel: Convergence and generalization in neural networks. *Advances in neural information processing systems* **31** (2018)
- [25] Wang, S., Wang, H., Perdikaris, P.: On the eigenvector bias of fourier feature networks: from regression to solving multi-scale pdes with physics-informed neural networks (2020) <https://doi.org/10.48550/arxiv.2012.10047>
- [26] Rahaman, N., Baratin, A., Arpit, D., Draxler, F., Lin, M., Hamprecht, F., Bengio, Y., Courville, A.: On the spectral bias of neural networks. In: Chaudhuri, K., Salakhutdinov, R. (eds.) *Proceedings of the 36th International Conference on Machine Learning*. *Proceedings of Machine Learning Research*, vol. 97, pp. 5301–5310. PMLR, ??? (2019). <https://proceedings.mlr.press/v97/rahaman19a.html>
- [27] Glorot, X., Bengio, Y.: Understanding the difficulty of training deep feedforward neural networks. In: *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, pp. 249–256 (2010)
- [28] Robbins, H., Monroe, S.: A stochastic approximation method. *The annals of mathematical statistics*, 400–407 (1951)
- [29] Goodfellow, I., Bengio, Y., Courville, A.: *Deep Learning*. MIT Press, ??? (2016)
- [30] Liu, D.C., Nocedal, J.: On the limited memory broyden–fletcher–goldfarb–shanno method. *Mathematical Programming* **45**(3), 503–528 (1989)

- [31] Luporini, F., Louboutin, M., Lange, M., Kukreja, N., Witte, P., Hückelheim, J., Yount, C., Kelly, P.H.J., Herrmann, F.J., Gorman, G.J.: Architecture and performance of devito, a system for automated stencil computation. *ACM Trans. Math. Softw.* **46**(1) (2020) <https://doi.org/10.1145/3374916>
- [32] Wang, S., Sankaran, S., Stinis, P., Perdikaris, P.: Simulating three-dimensional turbulence with physics-informed neural networks. *arXiv preprint arXiv:2507.08972* (2025)
- [33] Moreh, F., Lyu, H., Rizvi, Z.H., Wuttke, F.: Deep neural networks for crack detection inside structures. *Scientific Reports* **14**(1), 4439 (2024)